

CARTILHA TÉCNICA

Hospedagem e Publicação de Sites Institucionais Desenvolvidos com Next.js
Versão 1.0

1. Objetivo

Esta cartilha tem como objetivo orientar equipes de tecnologia da informação de prefeituras e órgãos municipais sobre o processo de publicação de um site desenvolvido em Next.js na internet.

Ao final deste documento, o leitor será capaz de:

- Registrar um domínio próprio;
- Escolher uma hospedagem adequada;
- Instalar Node.js;
- Configurar um projeto Next.js;
- Gerar uma versão estática em HTML quando necessário;
- Publicar o site em um servidor.

2. Conceitos Básicos

Antes de iniciar a publicação, é importante compreender alguns conceitos.

Domínio

O domínio é o endereço que os cidadãos utilizarão para acessar o site.
Exemplos:

- prefeitura.gov.br
- turismo.municipio.gov.br
- cultura.municipio.gov.br

Hospedagem

A hospedagem é o servidor onde os arquivos do site ficarão armazenados.
O servidor permanece ligado 24 horas por dia para que o site possa ser acessado pela internet.

Next.js

Next.js é uma tecnologia baseada em JavaScript utilizada para desenvolver aplicações web modernas. Dependendo da configuração do projeto, ele pode funcionar de duas formas:

Aplicação dinâmica

Necessita de Node.js rodando continuamente no servidor. Exemplo:

```
> npm start
```

Site estático

Gera arquivos HTML, CSS e JavaScript. Pode ser hospedado em qualquer servidor simples. Exemplos:

- Apache
- Nginx
- Hospedagens compartilhadas
- Servidores municipais

3. Escolhendo o Tipo de Servidor

Opção 1 – Servidor para Aplicações Node.js

Utilizar quando:

- O sistema possui funcionalidades dinâmicas;
- Há integração com banco de dados;
- Existem APIs.

Exemplos de servidores:

- Ubuntu Server
- Debian
- Windows Server

Requisitos mínimos:

- 2 CPUs
- 4 GB de RAM
- 40 GB de armazenamento

Opção 2 – Servidor para Sites Estáticos

Utilizar quando:

- O site possui apenas conteúdo informativo;
- Não há banco de dados;
- O projeto pode ser exportado para HTML.

Vantagens:

- Menor custo;
- Menor consumo de recursos;
- Maior simplicidade de manutenção.

4. Registro de Domínio

Órgãos públicos brasileiros normalmente utilizam domínios governamentais. Exemplos:

- prefeitura.gov.br
- turismo.municipio.gov.br
- educacao.municipio.gov.br

O registro deve seguir as orientações da infraestrutura de TIC responsável pelos domínios governamentais. Caso seja necessário registrar um domínio comum:

1. Acessar o Registro.br
2. Verificar disponibilidade do domínio
3. Efetuar o cadastro
4. Realizar o pagamento
5. Configurar os servidores DNS

Exemplo:

- www.municipio.com.br

5. Instalação do Node.js

Windows

Baixar o instalador LTS do Node.js.

Após a instalação, abrir o Prompt de Comando:

```
> node -v
```

Exemplo de resultado:

```
> v 22.15.0
```

Verificar também:

```
> npm -v
```

Ubuntu Server

Atualizar o sistema:

```
> sudo apt update  
> sudo apt upgrade -y
```

Instalar Node.js:

```
> sudo apt install nodejs npm -y
```

Verificar:

```
> node -v  
> npm -v
```

6. Obtendo o Projeto

Caso o projeto esteja armazenado em um repositório Git:

```
> git clone https://repositorio/projeto.git
```

Entrar na pasta:

```
> cd projeto
```

7. Instalação das Dependências

Todo projeto Next.js possui um arquivo chamado: *package.json*. Ele contém todas as bibliotecas necessárias.

Instalar as dependências:

```
> npm install
```

O sistema baixará automaticamente todos os pacotes necessários.

8. Executando o Projeto Localmente

Executar:

```
> npm run dev
```

A aplicação será iniciada normalmente em: *http://localhost:3000*

9. Gerando a Versão de Produção

Para preparar o projeto para publicação:

```
> npm run build
```

O Next.js criará uma versão otimizada. Em seguida:

```
> npm start
```

A aplicação será executada em modo de produção.

10. Publicando um Projeto Next.js Dinâmico

Após gerar a build:

```
> npm run build  
> npm start
```

É recomendável utilizar um gerenciador de processos.

11. Exportando o Site para HTML

Alguns servidores aceitam apenas arquivos HTML, CSS e JavaScript. Nesses casos, o projeto deve ser exportado como site estático.

Configuração do next.config.js

Adicionar:

```
/** @type {import('next').NextConfig} */  
const nextConfig = {  
  output: 'export',  
};  
  
module.exports = nextConfig;
```

Gerar a Build Estática

Executar:

```
> npm run build
```

O Next.js criará uma pasta chamada: *out*. Exemplo:

```
out/  
├── index.html  
├── sobre.html  
├── contato.html  
├── _next/  
├── imagens/  
└── css/
```

12. Publicando um Site HTML

Após gerar a pasta: *out*. Copiar todo o conteúdo para o servidor.

Exemplo Linux:

```
> scp -r out/* usuario@servidor:/var/www/html
```

Ou utilizando um cliente FTP:

- FileZilla
- WinSCP

13. Estrutura Final do Site no Servidor

Exemplo:

```
/var/www/html
├── index.html
├── sobre.html
├── contato.html
├── _next
├── imagens
└── css
```

14. Configuração de DNS

Após a contratação da hospedagem, será necessário apontar o domínio para o servidor.

Exemplo:

Tipo: A

Nome: @

Valor: 200.100.50.10

Para subdomínios:

Tipo: CNAME

Nome: turismo

Valor: municipio.gov.br

A propagação normalmente ocorre entre alguns minutos e 24 horas.

15. Checklist de Publicação

Antes de colocar o site em produção, verificar:

- ☐ Domínio registrado
- ☐ DNS configurado
- ☐ Node.js instalado
- ☐ Dependências instaladas
- ☐ Build executada sem erros
- ☐ Testes realizados
- ☐ Backup realizado
- ☐ HTTPS configurado
- ☐ Site acessível pela internet

16. Recomendações de Segurança

- Utilizar HTTPS;
- Manter Node.js atualizado;
- Realizar backups periódicos;
- Restringir acesso administrativo;
- Utilizar senhas fortes;
- Atualizar dependências regularmente;
- Monitorar disponibilidade do site.

Fluxo Resumido

1. Registrar domínio
- ↓
2. Contratar/configurar hospedagem
- ↓
3. Instalar Node.js
- ↓
4. Copiar projeto Next.js
- ↓
5. Executar npm install
- ↓
6. Executar npm run build
- ↓
- 7A. npm start (site dinâmico)
OU
- 7B. output:'export' + pasta out (site estático)
- ↓
8. Configurar DNS
- ↓
9. Publicar site

17. Hospedagem Recomendada para Municípios de Pequeno Porte

A escolha da hospedagem deve considerar o orçamento disponível, a capacidade técnica da equipe e a finalidade do site.

Cenário 1 – Site Institucional Simples

Indicado para:

- Portal de turismo;
- Portal cultural;
- Página institucional;
- Divulgação de eventos;
- Informações públicas.

Características:

- Sem banco de dados;
- Sem área administrativa complexa;
- Atualizações esporádicas.

Recomendação:

- Exportar o projeto Next.js para HTML estático;
- Hospedar em servidor web simples.

Vantagens:

- Menor custo;
- Menor necessidade de manutenção;
- Maior estabilidade.

Cenário 2 – Portal com Atualizações Frequentes

Indicado para:

- Notícias;
- Transparência;
- Serviços digitais;
- Integração com sistemas municipais.

Características:

- Banco de dados;
- APIs;
- Conteúdo atualizado constantemente.

Recomendação:

- Hospedagem VPS Linux;
- Aplicação Next.js executando com Node.js.

Cenário 3 – Hospedagem em Nuvem

Indicado para:

- Equipes sem infraestrutura própria;
- Projetos que exigem alta disponibilidade.

Opções:

- Vercel
- Netlify
- Railway
- DigitalOcean
- AWS
- Google Cloud

Vantagens:

- Atualizações simplificadas;
- Escalabilidade;
- Menor necessidade de administração de servidores.

Desvantagens:

- Dependência de fornecedores externos;
- Custos podem aumentar conforme o crescimento do projeto.

18. Comparativo de Custos

Hospedagem Compartilhada

Faixa estimada:

R\$ 15 a R\$ 50 por mês

Indicada para:

- Sites HTML estáticos.

VPS Linux

Faixa estimada:

R\$ 30 a R\$ 150 por mês

Indicada para:

- Aplicações Next.js com Node.js.

Servidor Próprio da Prefeitura

Custos:

- Aquisição de hardware;
- Energia elétrica;
- Backup;
- Manutenção;
- Segurança.

Indicado quando:

- Existe equipe técnica especializada;
- Há necessidade de manter dados localmente.

19. Fluxo de Atualização do Site

Sempre que houver uma nova versão do projeto:

1. Receber os arquivos da equipe de desenvolvimento;
2. Realizar backup da versão atual;
3. Atualizar os arquivos do servidor;
4. Executar testes básicos;
5. Validar acesso público.

Para projetos Next.js:

```
> git pull  
> npm install  
> npm run build  
> pm2 restart portal-municipal
```

Para sites HTML:

Substituir os arquivos da pasta publicada.

20. Procedimento de Backup

Recomenda-se:

Backup Diário

- Banco de dados;
- Arquivos enviados por usuários.

Backup Semanal

- Código-fonte;
- Configurações do servidor.

Backup Mensal

- Cópia completa do ambiente.

Os backups devem ser armazenados em local diferente do servidor principal.

21. Certificado Digital HTTPS

Todo portal público deve utilizar HTTPS.

Benefícios:

- Proteção dos dados transmitidos;
- Maior confiança dos cidadãos;
- Melhor posicionamento em mecanismos de busca.

Soluções recomendadas:

- Let's Encrypt (gratuito)
- Certificados corporativos quando exigidos pela política do órgão.

22. Responsabilidades da Equipe Municipal

Gestor do Contrato

Responsável por:

- Acompanhar fornecedores;
- Aprovar entregas;
- Fiscalizar a execução contratual.

Equipe de TI

Responsável por:

- Publicação do sistema;
- Backup;
- Atualizações;
- Monitoramento.

Fornecedor de Desenvolvimento

Responsável por:

- Entrega do código-fonte;
- Documentação técnica;
- Correção de falhas previstas contratualmente.

23. Exigências Mínimas na Entrega do Projeto

Todo fornecedor contratado deve entregar:

- ✓ Código-fonte completo
- ✓ Manual técnico
- ✓ Manual do usuário
- ✓ Arquivos de instalação
- ✓ Banco de dados (quando aplicável)
- ✓ Credenciais administrativas
- ✓ Arquitetura da solução
- ✓ Inventário de dependências
- ✓ Procedimento de atualização
- ✓ Procedimento de backup

A ausência desses itens pode dificultar a manutenção futura do sistema pelo município.

25. Alimentação de hotéis no site

O site foi projetado para não precisar de uma área administrativa, banco de dados ou back-end para alimentar o site, por isso para atualizar a parte de hotéis disponíveis, precisa-se de uma pessoa com o mínimo de conhecimentos em código para seguir as seguintes instruções e inserir as informações manual e diretamente no código fonte.

Compenente *StaySection*

No arquivo principal do projeto, *page.js*, na linha 459, inicia-se a secção de hotéis com componentes reutilizáveis, para adicionar mais hotéis ou editar os já existentes é preciso inserir ou alterar as informações dentro do array dentro da propriedade *cards* dentro do componente *StaySection*.

Função de cada parâmetro dentro da propriedade *card*:

- **id**: Número de identificação do card;
- **title**: Título do card, nome do hotel;
- **description**: Descrição sobre o hotel;
- **image**: imagem do hotel (**ATENÇÃO**: SEMPRE SALVAR AS IMAGENS DIRETAMENTE NO DIRETÓRIO PUBLIC);
- **rating**: Nota/Avaliação do hotel,

Caso precise alterar o site onde será o formulário de preenchimento para os hotéis que se inscreverem, modifique o parâmetro *link* dentro da propriedade *cta*.

```
459 <section className={styles.hoteis}>
460   <StaySection
461     bannerTitle="ONDE FICAR EM DIANÓPOLIS"
462     bannerSubtitle="Município de Dianópolis de Serras Gerais"
463     bannerImage="bannerhoteis.jpg"
464     cards=[
465       {
466         id: "1",
467         title: "Pousada Exemplo",
468         description: "Área, ar e café da manhã incluso.",
469         image: "pousadaexemplo1.jpg",
470         rating: 4.5,
471       },
472       {
473         id: "2",
474         title: "Pousada Exemplo",
475         description: "Ótima localização e conforto.",
476         image: "pousadaexemplo2.jpg",
477         rating: 4.5,
478       },
479       {
480         id: "3",
481         title: "Pousada Exemplo",
482         description: "Espaço amplo e estacionamento.",
483         image: "pousadaexemplo3.jpg",
484         rating: 4.5,
485       },
486     ]
487     cta={
488       title: "Seu estabelecimento aqui?",
489       description: "Faça parte do guia oficial.",
490       buttonText: "Cadastrar",
491       link: "https://forms.gle/YjZhSd96eHy38R878",
492     }
493   />
494 </section>
```